Porting ROSPlan from ROS1 to ROS2 Humble: A Practical Guide

This document describes the complete migration of ROSPlan — a task planning framework for ROS — from ROS1 (catkin, C++14, Ubuntu 20.04) to ROS2 Humble Hawksbill (ament_cmake, C++17, Ubuntu 22.04). The migration covers nine packages totalling approximately 77 000 lines of C++ and 6 000 lines of Python. Each section documents a category of change, the underlying reason, and the specific solution applied. The goal is to provide a reusable reference for teams migrating similarly structured ROS1 frameworks.

Table of Contents

1. Package Overview
2. Build System
3. Node Initialization and Architecture
4. Parameters
5. Service Servers and Clients
6. Publishers, Subscribers, and QoS
7. Asynchronous Service Calls and Executor Patterns
8. Topic Namespace Resolution
9. Time, Duration, and Timers
10. Logging
11. Action Servers and Clients
12. Python Migration (rospy → rclpy)
13. Launch Files (XML → Python)
14. Message and Service Types
15. VAL PDDL Parser Integration
16. Compiler and Platform Issues (GCC 11 / Ubuntu 22.04)
17. Integration Testing: Bugs Found and Fixed
18. End-to-End Validation

---

1. Package Overview

ROSPlan is organized as a ROS meta-package. The migration order follows the dependency graph, starting with packages that have no ROS API surface and finishing with the GUI layer:

Package	Language	Dependency	Notes
`rosplan_dependencies`	C++	—	VAL PDDL library, HDDL parser, CHIMP planner
`rosplan_knowledge_msgs`	IDL	—	Messages, services, actions
`rosplan_dispatch_msgs`	IDL	—	Messages, services, actions
`rosplan_knowledge_base`	C++	msgs	Largest node; establishes all patterns
`rosplan_planning_system`	C++	KB + msgs	20 executables; planner adapters, dispatchers
`rosplan_action_interface`	Python	dispatch_msgs	Base classes for action execution
`rosplan_sensing_interface`	Python	knowledge_msgs	Periodic KB updates from sensors
`rosplan_rqt`	Python/Qt	all	rqt GUI plugins
`rosplan`	—	all	Meta-package; trivial

Migration is done package by package with `colcon build --packages-select <pkg>` to verify each step before proceeding.

---

2. Build System

2.1 package.xml

Every `package.xml` is updated from format 1 (catkin) to format 3 (ament):

```
<!-- ROS1 -->
<package>
  <buildtool_depend>catkin</buildtool_depend>
  <build_depend>roscpp</build_depend>
  <run_depend>roscpp</run_depend>
</package>

<!-- ROS2 -->
<package format="3">
  <buildtool_depend>ament_cmake</buildtool_depend>
  <depend>rclcpp</depend>   <!-- combines build + exec -->
</package>
```

Key dependency replacements:

ROS1	ROS2
`actionlib`	`rclcpp_action`
`tf`	`tf2`, `tf2_ros`
`actionlib_msgs`	`action_msgs` (built-in)
`roscpp`	`rclcpp`
`rospy`	`rclpy`

2.2 CMakeLists.txt

```
# ROS1 skeleton
cmake_minimum_required(VERSION 2.8.3)
find_package(catkin REQUIRED COMPONENTS roscpp ...)
catkin_package(...)
include_directories(${catkin_INCLUDE_DIRS})
target_link_libraries(my_node ${catkin_LIBRARIES})

# ROS2 skeleton
cmake_minimum_required(VERSION 3.8)
set(CMAKE_CXX_STANDARD 17)
find_package(ament_cmake REQUIRED)
find_package(rclcpp REQUIRED)
...
ament_target_dependencies(my_node rclcpp ...)
install(TARGETS my_node DESTINATION lib/${PROJECT_NAME})
ament_package()
```

All packages move from C++14 to C++17 (`set(CMAKE_CXX_STANDARD 17)`).

Targets and headers must be explicitly installed in ROS2; the catkin devel-space is gone:

```
install(TARGETS my_node     DESTINATION lib/${PROJECT_NAME})
install(DIRECTORY include/  DESTINATION include)
install(DIRECTORY launch/   DESTINATION share/${PROJECT_NAME}/launch)
install(DIRECTORY common/   DESTINATION share/${PROJECT_NAME}/common)
```

Trailing-slash semantics in `install(DIRECTORY ...)`: `install(DIRECTORY src DESTINATION dst)` copies the directory `src` itself into `dst`, producing `dst/src/`. `install(DIRECTORY src/ DESTINATION dst)` copies the *contents* of `src` into `dst`. This distinction caused a double `bin/bin/` path for the POPF binary and is easy to overlook.

2.3 Message and interface generation

```
# ROS1
generate_messages(DEPENDENCIES std_msgs ...)
add_dependencies(my_node ${catkin_EXPORTED_TARGETS})

# ROS2
rosidl_generate_interfaces(${PROJECT_NAME}
  "msg/Foo.msg"
  "srv/Bar.srv"
  "action/Baz.action"
  DEPENDENCIES std_msgs builtin_interfaces diagnostic_msgs
)
```

The `actionlib_msgs` package no longer exists; ROS2 action definitions do not include a header from it — the `action_msgs` dependency is automatic via `rosidl_generate_interfaces`.

---

3. Node Initialization and Architecture

3.1 From NodeHandle to Node inheritance

The fundamental shift in ROS2 is that a node *is* an object, not a handle to an external context:

```
// ROS1: node handle injected into the class
class KnowledgeBase {
    ros::NodeHandle _nh;
public:
    KnowledgeBase(ros::NodeHandle& nh) : _nh(nh) {}
};

int main(int argc, char** argv) {
    ros::init(argc, argv, "rosplan_knowledge_base");
    ros::NodeHandle nh("~");
    KnowledgeBase kb(nh);
    ros::spin();
}

// ROS2: class inherits from rclcpp::Node
class KnowledgeBase : public rclcpp::Node {
public:
    KnowledgeBase() : Node("rosplan_knowledge_base") {}
};

int main(int argc, char** argv) {
    rclcpp::init(argc, argv);
    auto node = std::make_shared<KnowledgeBase>();
    rclcpp::executors::MultiThreadedExecutor exec;
    exec.add_node(node);
    exec.spin();
    rclcpp::shutdown();
}
```

All service registrations, subscriber creations, and parameter declarations that previously took a `NodeHandle` reference now become member-function calls (`create_service<>()`, `create_subscription<>()`, `declare_parameter()`) inside the class.

3.2 Executor model

`MultiThreadedExecutor` is used for all nodes that contain service servers. This allows the executor to call multiple service callbacks concurrently and prevents head-of-line blocking.

Two callback groups are created in each node constructor:

```
srv_cbg_ = create_callback_group(rclcpp::CallbackGroupType::MutuallyExclusive);
sub_cbg_ = create_callback_group(rclcpp::CallbackGroupType::Reentrant);
```

Service servers are assigned to `srv_cbg_` and topic subscriptions to `sub_cbg_`. The `MutuallyExclusive` group prevents concurrent calls to the same callback; `Reentrant` allows the dispatcher to receive multiple action-feedback messages concurrently.

3.3 Constructor error-exit guard

Nodes that detect a fatal configuration error (e.g., a missing domain file) call `rclcpp::shutdown()` from their constructor. Without a guard, `exec.add_node()` will throw an `RCLError` on an invalid context:

```
auto node = std::make_shared<KnowledgeBase>();
if (!rclcpp::ok()) return 1;   // constructor called shutdown on error
rclcpp::executors::MultiThreadedExecutor exec;
exec.add_node(node);
exec.spin();
```

Applied to `KnowledgeBase` and `ProblemInterface`.

---

4. Parameters

```
// ROS1
double rate;
nh.param<double>("kb_rate", rate, 10.0);

// ROS2: declare before first access
declare_parameter("kb_rate", 10.0);
double rate = get_parameter("kb_rate").as_double();
```

Key rules: - Parameters must be declared before `get_parameter()` is called; there is no implicit declaration on first access. - The `~` prefix for private parameters in ROS1 is replaced by declaring inside the Node class (parameters automatically scope to the node's namespace).

---

5. Service Servers and Clients

5.1 Server

```
// ROS1
_nh.advertiseService("domain/name", &KnowledgeBase::getDomainName, this);
bool getDomainName(rosplan_knowledge_msgs::GetDomainName::Request& req,
                   rosplan_knowledge_msgs::GetDomainName::Response& res) { ... return true

// ROS2
create_service<rosplan_knowledge_msgs::srv::GetDomainNameService>(
    "~/domain/name",
    [this](const std::shared_ptr<...::Request> req, std::shared_ptr<...::Response> res) {
        getDomainName(req, res);
    },
    rmw_qos_profile_services_default,
    srv_cbg_);
void getDomainName(const std::shared_ptr<...::Request>, std::shared_ptr<...::Response>) {
```

The callback return type changes from `bool` to `void`. `ros::ServiceEvent<Req,Res>` — used in ROS1 for caller-IP introspection — has no ROS2 equivalent; all event-based tracking is removed.

5.2 Service namespace

In ROS1, service names are resolved globally from the node's private namespace (`~`). In ROS2 a relative name `"domain/name"` resolves to `/domain/name` (root namespace). The planning nodes build client paths as `"/" + kb_name + "/domain/name"` (e.g., `/rosplan_knowledge_base/domain/name`). To match, KB services must use the `~/` prefix:

```
// Wrong in ROS2 — resolves to /domain/name, not /rosplan_knowledge_base/domain/name
create_service<T>("domain/name", ...);

// Correct — ~/  expands to /<namespace>/<node_name>/
create_service<T>("~/domain/name", ...);
```

All 19 KB service and publisher registrations were updated this way.

---

6. Publishers, Subscribers, and QoS

6.1 Latched publishers

```
// ROS1: third argument enables latching
ros::Publisher pub = nh.advertise<T>("topic", 1, true);

// ROS2: QoS policy
auto pub = create_publisher<T>("topic", rclcpp::QoS(1).transient_local());
```

6.2 Latched subscribers — both sides must match

ROS2/DDS transient-local delivery requires **both** publisher and subscriber to declare `transient_local()`. A subscriber with default (volatile) durability does **not** receive stored messages from a transient-local publisher — it only receives messages published after it connected.

This caused two silent failures during integration testing:

Subscriber node	Topic	Publisher QoS	Subscriber QoS (broken)
`PDDLSimplePlanParser`	`planner_output`	transient_local	default (volatile)
`SimplePlanDispatcher`	`complete_plan`	transient_local	default (volatile)

Both components were started after the publisher had already published; with volatile QoS they never received the stored message.

Fix:

```
// PDDLSimplePlanParser.cpp
planner_output_sub_ = create_subscription<std_msgs::msg::String>(
    planner_topic, rclcpp::QoS(1).transient_local(),
    [this](const std_msgs::msg::String::SharedPtr msg) { plannerCallback(msg); },
    sub_opts);

// SimplePlanDispatcher.cpp
plan_sub_ = create_subscription<rosplan_dispatch_msgs::msg::CompletePlan>(
    plan_topic, rclcpp::QoS(1).transient_local(),
    [this](const rosplan_dispatch_msgs::msg::CompletePlan::SharedPtr msg) { planCallback(m
    sub_opts);
```

7. Asynchronous Service Calls and Executor Patterns

7.1 The core problem

ROS2 uses an async service-call model: `async_send_request()` returns a future; the response arrives via a callback processed by the executor. Blocking on the future with `wait_for()` from a thread that is part of (or is blocking) the executor will deadlock:

```
// Deadlock if called before exec.spin() or from inside a callback
// that holds the executor lock:
auto future = client->async_send_request(req);
future.wait_for(std::chrono::seconds(10));   // executor not spinning → never resolves
```

Three places in ROSPlan were affected.

7.2 Setup-time calls (before executor starts): satellite node

`RPActionInterface::setup()` is called from `main()` before `exec.spin()`. It needs to query the KB for operator and predicate details.

Pattern: Create a temporary node used only for this call, then spin it:

```
auto svc_node = std::make_shared<rclcpp::Node>("rp_action_setup_svc_node");
auto op_client = svc_node->create_client<GetDomainOperatorDetailsService>(svc_name);
op_client->wait_for_service(std::chrono::seconds(20));

auto future = op_client->async_send_request(req);
if (rclcpp::spin_until_future_complete(svc_node, future, std::chrono::seconds(10))
        != rclcpp::FutureReturnCode::SUCCESS) {
    RCLCPP_ERROR(get_logger(), "KB operator_details call failed");
    return;
}
auto res = future.get();
```

The satellite node is ephemeral (local variable); it is never added to any executor and is destroyed at the end of `setup()`.

For the KB update calls in `dispatchCallback()`, a persistent satellite node is needed because those calls happen from inside an executor callback thread (and the `update_knowledge_client_` must outlive setup):

```
// In RPActionInterface.h:
rclcpp::Node::SharedPtr kb_svc_node_;
rclcpp::Client<KnowledgeUpdateServiceArray>::SharedPtr update_knowledge_client_;

// In setup():
kb_svc_node_ = std::make_shared<rclcpp::Node>("rp_action_kb_svc_node");
update_knowledge_client_ = kb_svc_node_->create_client<KnowledgeUpdateServiceArray>(
    "/" + kb + "/update_array");

// In dispatchCallback():
auto future = update_knowledge_client_->async_send_request(req);
rclcpp::spin_until_future_complete(kb_svc_node_, future, std::chrono::seconds(5));
```

7.3 Calls from non-executor dispatch threads: future wait

The plan dispatcher runs `dispatchPlan()` from a detached thread (not an executor callback). Because this thread does not hold an executor lock, it is safe to use a simple `wait_for`:

```
template<typename ClientT, typename ReqT, typename ResT>
bool callServiceSync(std::shared_ptr<ClientT>& client,
                     std::shared_ptr<ReqT> req,
                     std::shared_ptr<ResT>& res)
{
    if (!client->wait_for_service(std::chrono::seconds(5))) return false;
    auto future = client->async_send_request(req);
    if (future.wait_for(std::chrono::seconds(5)) != std::future_status::ready) return fals
    res = future.get();
    return true;
}
```

This template is declared inline in the protected section of `PlanDispatcher` and `PlanParser` base classes so all subclasses inherit it without a satellite node.

Decision rule:

Call site	Executor running?	Holds executor lock?	Safe pattern
`main()` before `exec.spin()`	No	No	Satellite node + `spin_until_future_complete`
Inside executor callback	Yes	Yes	Satellite node + `spin_until_future_complete`
Detached / non-executor thread	Yes	No	Plain `wait_for`

---

8. Topic Namespace Resolution

In ROS2, topic name resolution follows these rules:

Prefix	Example	Resolved path
None (relative)	`foo`	`/<node_namespace>/foo`
Tilde (node-private)	`~/foo`	`/<node_namespace>/<node_name>/foo`
Leading slash (absolute)	`/foo`	`/foo`

With the default namespace `/`, a relative name `foo` becomes `/foo`, **not** `/<node_name>/foo`. The node name plays no role in relative name resolution.

This caused a topic mismatch during integration testing: the planner interface published to `planner_output` (relative → `/planner_output`), while a test script subscribed to `/rosplan_planner_interface/planner_output` (incorrectly treating the node name as a namespace prefix). The fix was to use the same relative name `planner_output` on both ends.

---

9. Time, Duration, and Timers

```
// ROS1
ros::Time now = ros::Time::now();
ros::Duration d(1.5);
ros::Rate rate(10);  // 10 Hz
while (ros::ok()) { ...; rate.sleep(); }

// ROS2
rclcpp::Time now = this->now();              // inside Node
rclcpp::Time now = rclcpp::Clock().now();   // static context
rclcpp::Duration d = rclcpp::Duration::from_seconds(1.5);

// Spin loops replaced by wall timers:
timer_ = create_wall_timer(
    std::chrono::milliseconds(100),          // 10 Hz
    std::bind(&MyNode::timerCallback, this));
```

When comparing `builtin_interfaces/Time` message fields arithmetically, wrap them:

```
rclcpp::Time msg_time(msg.header.stamp);
auto delta = this->now() - msg_time;
```

10. Logging

```
// ROS1
ROS_INFO("value: %d", x);
ROS_WARN("warning");

// ROS2 — inside a Node
RCLCPP_INFO(get_logger(), "value: %d", x);
RCLCPP_WARN(get_logger(), "warning");

// ROS2 — outside a Node (e.g., free function)
RCLCPP_INFO(rclcpp::get_logger("my_logger"), "value: %d", x);
```

11. Action Servers and Clients

```
// ROS1
#include <actionlib/server/simple_action_server.h>
actionlib::SimpleActionServer<PlanAction> server_(nh, "name", executeCallback, false);

// ROS2
#include <rclcpp_action/rclcpp_action.hpp>
rclcpp_action::Server<PlanAction>::SharedPtr server_ =
    rclcpp_action::create_server<PlanAction>(
        this, "name",
        [this](auto uuid, auto goal)  { return handle_goal(uuid, goal);  },
        [this](auto goal_handle)      { return handle_cancel(goal_handle); },
        [this](auto goal_handle)      { handle_accepted(goal_handle);     });
```

The single `execute` callback splits into three: `handle_goal` (accept/reject), `handle_cancel`, and `handle_accepted`. The accepted callback must not block — it launches a detached thread:

```
void handle_accepted(std::shared_ptr<GoalHandle> gh) {
    std::thread([this, gh]() { executePlan(gh); }).detach();
}
```

Goal result reporting:

```
// ROS1
server_.setSucceeded(result);
server_.setAborted(result);

// ROS2
goal_handle->succeed(result);
goal_handle->abort(result);
```

12. Python Migration (rospy → rclpy)

12.1 Core API table

ROS1 (rospy)	ROS2 (rclpy)
`import rospy`	`import rclpy` from `rclpy.node import Node`
`rospy.init_node("name")`	`rclpy.init(); node = Node("name")`
`rospy.get_param("name", default)`	`node.declare_parameter("name", default); node.get_parameter("name").value`
`rospy.loginfo(msg)`	`node.get_logger().info(msg)`
`rospy.ServiceProxy("name", Type)`	`node.create_client(Type, "name")`
`rospy.wait_for_service("name")`	`client.wait_for_service()`
`rospy.spin()`	`rclpy.spin(node)`
`rospkg.RosPack().get_path("pkg")`	`ament_index_python.packages.get_package_share_directory("pkg")`
`rospy.Subscriber(t, T, cb, arg)`	`node.create_subscription(T, t, lambda msg, a=arg: cb(msg, a), qos)`

In ROS1, `rospy` functions are module-level globals. In ROS2 every ROS operation requires an explicit `Node` instance. The top-level class inherits `rclpy.Node`; helper classes receive it as a constructor argument and store it as `self._node`.

12.2 Blocking service calls (Python)

The same deadlock risk exists in Python. The satellite node pattern:

```
self._svc_node = Node('helper_svc_node')  # NOT added to any executor
client = self._svc_node.create_client(SrvType, '/topic')

def call_blocking(client, request, timeout_sec=5.0):
    if not client.wait_for_service(timeout_sec=timeout_sec):
        return None
    future = client.call_async(request)
    rclpy.spin_until_future_complete(self._svc_node, future, timeout_sec=timeout_sec)
    return future.result() if future.done() else None
```

Applied in `RPKnowledgeBaseLink`, `ServiceActionInterface`, and all `rosplan_rqt` widgets.

12.3 Action clients (Python)

`actionlib.SimpleActionClient.send_goal(goal, done_cb=cb)` becomes an async chain:

```
send_future = client.send_goal_async(goal)
send_future.add_done_callback(lambda f: self._on_goal_accepted(f))

def _on_goal_accepted(self, future):
    goal_handle = future.result()
    result_future = goal_handle.get_result_async()
    result_future.add_done_callback(lambda f: self._on_result(f.result().result))
```

Goal status constant: `actionlib.GoalStatus.SUCCEEDED` → `action_msgs.msg.GoalStatus.STATUS_SUCCEEDED`.

12.4 Rate loops → timers

```
# ROS1
rate = rospy.Rate(10)
while not rospy.is_shutdown():
    self.tick()
    rate.sleep()

# ROS2
self.create_timer(0.1, self._tick)  # in __init__
```

12.5 Complex configuration parameters

ROS2 parameters cannot hold nested dicts or lists-of-dicts. Configuration that was previously loaded via `rosparam load` (YAML into the parameter server) is now read from a YAML file whose path is passed as a plain string parameter:

```
self.declare_parameter('config_path', '')
config_path = self.get_parameter('config_path').value
with open(config_path) as f:
    config = yaml.safe_load(f)
```

```
ros2 run rosplan_sensing_interface sensing_interface \
    --ros-args -p config_path:=/path/to/config.yaml
```

The YAML structure is unchanged from ROS1.

12.6 Python 2 → Python 3

Pattern	Fix
`dict.iteritems()`	`dict.items()`
`reduce(f, lst)`	`from functools import reduce`
`e.message`	`str(e)`
`imp.load_source('n', path)`	`importlib.util.spec_from_file_location` + `exec_module`
`from string import join, split`	`' '.join(lst)`, `str.split()`
`except E, e:`	`except E as e:`
`print "x"`	`print('x')`

12.7 Dynamic message import

```
# ROS1
exec('from pkg.msg import FooAction', globals())
goal = FooGoal()

# ROS2
action_module = importlib.import_module('pkg.action')
ActionType = getattr(action_module, 'Foo')
goal = ActionType.Goal()
```

12.8 KB update_type field

```
# ROS1 — bytes concatenation
kus.update_type += np.array(kus.ADD_KNOWLEDGE).tostring()

# ROS2 — int32[] maps to a Python list
kus.update_type.append(KnowledgeUpdateServiceArray.Request.ADD_KNOWLEDGE)
```

---

13. Launch Files (XML → Python)

13.1 Conversion table

ROS1 (XML)	ROS2 (Python)
`<arg name="X" default="Y"/>`	`DeclareLaunchArgument('X', default_value='Y')`
`$(arg X)`	`LaunchConfiguration('X')`
`$(find pkg)`	`PathJoinSubstitution([FindPackageShare('pkg'), ...])`
`<node pkg="P" type="T" name="N">`	`Node(package='P', executable='T', name='N', output='screen')`
`<param name="K" value="V"/>`	`parameters=[{'K': LaunchConfiguration('V')}]`
`<include file="..."/>`	`IncludeLaunchDescription(PythonLaunchDescriptionSource(...))`
`<remap from="a" to="b"/>`	`remappings=[('a', 'b')]` inside `Node()`
`<rosparam command="load" file="..."/>`	`parameters=[{'config_path': PathJoinSubstitution([...])}]`

13.2 Dynamic node names from arguments

```
<!-- ROS1 -->
<node name="rosplan_interface_$(arg pddl_action_name)" .../>
```

```
# ROS2 — pass a list
Node(name=['rosplan_interface_', LaunchConfiguration('pddl_action_name')], ...)
```

13.3 Path resolution: share directory

In ROS2, `FindPackageShare('pkg')` resolves to the installed `share/pkg/` directory (not the source tree). All data files must be installed:

```
install(DIRECTORY common/bin/  DESTINATION share/${PROJECT_NAME}/common/bin USE_SOURCE_PERI
install(DIRECTORY test/        DESTINATION share/${PROJECT_NAME}/test)
install(DIRECTORY launch/      DESTINATION share/${PROJECT_NAME}/launch)
```

13.4 Default values containing paths

```
DeclareLaunchArgument(
    'planner_command',
    default_value=[
        'timeout 10 ',
        PathJoinSubstitution([FindPackageShare('rosplan_planning_system'),
                              'common', 'bin', 'popf']),
        ' DOMAIN PROBLEM',
    ]
)
```

---

14. Message and Service Types

14.1 Header names

```
// ROS1
#include "rosplan_knowledge_msgs/KnowledgeItem.h"

// ROS2 — snake_case, .hpp, under msg/ or srv/ subdirectory
#include "rosplan_knowledge_msgs/msg/knowledge_item.hpp"
```

14.2 Namespace

```
// ROS1
rosplan_knowledge_msgs::KnowledgeItem item;
rosplan_knowledge_msgs::GetDomainNameService srv;

// ROS2
rosplan_knowledge_msgs::msg::KnowledgeItem item;
rosplan_knowledge_msgs::srv::GetDomainNameService srv;
```

14.3 Request and response access

```
// ROS2 nested types
auto req = std::make_shared<MySrv::Request>();
req->field = value;
auto future = client->async_send_request(req);
auto res = future.get();
bool ok = res->success;
```

---

15. VAL PDDL Parser Integration

ROSPlan embeds two copies of the VAL PDDL validation library with incompatible namespaces:

Location	Namespace	Used by
`rosplan_dependencies/VAL/`	`VAL`	`libval_kcl.so` (external planners)
`rosplan_knowledge_base/src/VALfiles/`	`VAL1_2`	`knowledgeBase` executable

The two copies **cannot be linked together**. The knowledge base compiles its VALfiles directly into the executable and does not link `libval_kcl`.

15.1 FlexLexer.h API version

The pre-generated PDDL flex scanner embedded in `pddl+.cpp` was built against an older `FlexLexer.h` API that uses `std::istream*` (pointer). Ubuntu 22.04's system header at `/usr/include/FlexLexer.h` has been updated to use `std::istream&` (reference).

The solution is to keep a copy of the pointer-API header alongside the scanner source:

```
rosplan_knowledge_base/src/VALfiles/FlexLexer.h
```

copied from `rosplan_dependencies/VAL/include/FlexLexer.h`.

15.2 HDDL parser: shared vs. static library

Both the PDDL scanner and the HDDL scanner (`rosplan_dependencies`) define the global flex memory functions `yyalloc`, `yyrealloc`, and `yyfree`. When `libhddl_parser` is a static library, these symbols collide at final link time.

Building it as a **shared library** (`.so`) isolates the symbol scope and resolves the conflict:

```
# rosplan_dependencies/CMakeLists.txt
add_library(hddl_parser SHARED ...)
```

16. Compiler and Platform Issues (GCC 11 / Ubuntu 22.04)

16.1 GCC 11 internal compiler error at -O1

GCC 11 produces an internal compiler error (segmentation fault in the compiler itself) when compiling files that include complex ROS2 template headers (`rcpputils/join.hpp`, `rclcpp/parameter_client.hpp`) at optimization level `-O1`.

Removing the explicit `-O1` flag from `target_compile_options` avoids the crash; GCC defaults to no optimization in Debug mode or `-O2` in Release mode, both of which work correctly.

16.2 Bison auxiliary header installation

Bison generates several auxiliary headers alongside the main `hddl_parser.tab.hpp`: `location.hh`, `position.hh`, `stack.hh`. These must be explicitly installed because `hddl_parser.h` includes them:

```
install(FILES
  ${CMAKE_CURRENT_BINARY_DIR}/hddl_parser/hddl_parser.tab.hpp
  ${CMAKE_CURRENT_BINARY_DIR}/hddl_parser/location.hh
  ${CMAKE_CURRENT_BINARY_DIR}/hddl_parser/position.hh
  ${CMAKE_CURRENT_BINARY_DIR}/hddl_parser/stack.hh
  DESTINATION include/rosplan_dependencies/hddl_parser)
```

17. Integration Testing: Bugs Found and Fixed

Basic unit and build-level tests pass early, but several issues only appear when the full pipeline is exercised end-to-end.

17.1 Knowledge base service namespace mismatch

Symptom: Every KB service call from the planning nodes timed out immediately after launch.

Root cause: KB services were registered with relative names (`"domain/name"`) which resolve to `/domain/name` (root namespace). Planning nodes construct client paths as `"/" + kb_name + "/domain/name"` = `/rosplan_knowledge_base/domain/name`.

Fix: Prefix all 19 KB service registrations in `KnowledgeBase.cpp` with `~/`:

```
create_service<T>("~/domain/name", ...);
// resolves to /rosplan_knowledge_base/domain/name ✓
```

17.2 POPF binary path double-nesting

Symptom: Planner node reported "No such file or directory" for the POPF binary.

Root cause: `install(DIRECTORY common/bin DESTINATION .../common/bin/)` (no trailing slash on source) copies the `bin/` directory itself into the destination, producing the path `share/rosplan_planning_system/common/bin/bin/popf`.

Fix: Add trailing slash to source path:

```
install(DIRECTORY common/bin/
  DESTINATION share/${PROJECT_NAME}/common/bin
  USE_SOURCE_PERMISSIONS)
```

17.3 `RPActionInterface::setup()` deadlock

Symptom: Every simulated action interface printed "could not call KB for operator details" immediately after startup.

Root cause: `setup()` is called from `main()` before `exec.spin()`. It called `async_send_request()` then `opFuture.wait_for(10s)`. Without a spinning executor, the response callback can never fire, so `wait_for` always times out.

Fix: Satellite node pattern (see Section 7.2).

17.4 Transient-local QoS mismatch

Symptom: Parser reported "No new planner output received; nothing to parse." Dispatcher reported "Dispatch complete" immediately with no actions executed.

Root cause: Both `PDDLSimplePlanParser` and `SimplePlanDispatcher` subscribed with default (volatile) QoS to topics published with `transient_local()`. Because the components were started sequentially, both subscribers connected after the publisher had already emitted the stored message.

Fix: Add `rclcpp::QoS(1).transient_local()` to both subscriptions (see Section 6.2).

17.5 `ros2 service call` has no `--spin-time` flag

`ros2 service call` in Humble does not support `--spin-time`. Test scripts using that flag fail immediately with exit code 2. Use the POSIX `timeout` utility instead:

```
timeout 15 ros2 service call /service_name pkg/srv/Type "{}"
```

---

18. End-to-End Validation

The full pipeline was validated with the turtlebot domain: 5 waypoints, one robot (`kenny`), goals to visit all waypoints and return docked.

Pipeline steps:

1. Start KB with `domain.pddl`
2. Load state via Python helper (15 items: 5 waypoints, 1 robot, 3 facts, 6 goals)
3. Generate problem file (32 lines, correct objects and goals)
4. Run POPF — produces 12-action plan in < 0.01 s

```
0.000: (undock kenny wp0) [2.0] 2.001: (localise kenny) [1.0] 3.002: (goto_waypoint kenny wp0 wp0) [1.0] ... 12.011: (dock kenny wp0) [3.0]
```

1. Parse plan — 12 `ActionDispatch` messages published on `complete_plan`
2. Start 4 simulated action interfaces (`undock`, `dock`, `localise`, `goto_waypoint`)
3. Dispatch — all 12 actions executed in ~18 s

KB state after dispatch:

- `visited(wp0)`, `visited(wp1)`, `visited(wp2)`, `visited(wp3)`, `visited(wp4)` — all present
- `docked(kenny)` — present
- `robot_at(kenny, wp0)` — present

Test result: 12/12 checks pass.

Summary

The table below summarises the primary change categories, effort estimate (relative), and the packages affected:

Category	Effort	Packages affected
Build system (package.xml, CMakeLists.txt)	Medium	All 9
Node inheritance / executor model	High	KB, planning_system
Service name namespace (`~/` prefix)	Low	KB
Async service call pattern	High	planning_system, action_interface
Transient-local QoS on subscribers	Low	planning_system
Python rospy → rclpy	High	action_interface, sensing_interface, rqt
Launch files XML → Python	Medium	KB, planning_system, sensing_interface
VAL FlexLexer / hddl_parser shared	Medium	dependencies, KB
GCC 11 -O1 ICE	Low	KB
Bison header installation	Low	dependencies

The migration preserves ROSPlan's full feature set and all nine packages build and pass end-to-end tests on ROS2 Humble / Ubuntu 22.04.